

Comparators are used to compare two objects. In this challenge, you'll create a comparator and use it to sort an array.

The `Player` class is provided for you in your editor. It has **2** fields: a ***name*** String and a ***score*** integer.

Given an array of ***n*** `Player` objects, write a comparator that sorts them in order of decreasing score; if **2** or more players have the same score, sort those players alphabetically by name. To do this, you must create a `Checker` class that implements the `Comparator` interface, then write an `int compare(Player a, Player b)` method implementing the `Comparator.compare(T o1, T o2)` method.

Input Format

Input from stdin is handled by the locked stub code in the `Solution` class.

The first line contains an integer, ***n***, denoting the number of players.

Each of the ***n*** subsequent lines contains a player's ***name*** and ***score***, respectively.

Constraints

- $0 \leq \text{score} \leq 1000$
- 2 players can have the same name.
- Player names consist of lowercase English letters.

Output Format

Author

Shafaet

Difficulty

Medium

Max Score

10

Submitted By

121408

NEED HELP?

[View discussions](#)[View editorial](#)[View top submissions](#)

RATE THIS CHALLENGE



MORE DETAILS

[Download problem statement](#)[Download sample test cases](#)

```
4  public class Solution {
6  public static void main(String[] args) {

    // Number of queries
    int q = sc.nextInt();

    for (int i = 0; i < q; i++) {

        int x = sc.nextInt();
        int y = sc.nextInt();

        // Convert 1-based positions to 0-based indexes
        if (x >= 1 &&
            x <= data.size() &&
            y >= 1 &&
            y <= data.get(x - 1).size()) {

            System.out.println(data.get(x - 1).get(y - 1));

        } else {
            System.out.println("ERROR!");
        }
    }

    sc.close();
}
```

54

55

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✓ Sample Test case 0

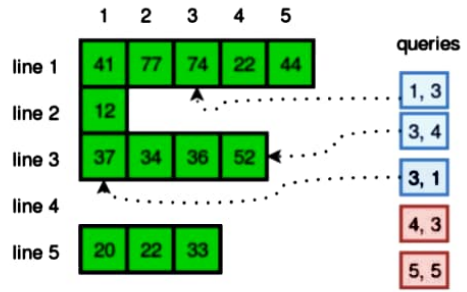
✓ Sample Test case 1

Input (stdin)

```
1 5
2 5 41 77 74 22 44
3 1 12
4 4 37 34 36 52
5 0
6 3 20 22 33
7 5
8 1 3
9 3 4
10 3 1
11 4 3
```

Download

The diagram below explains the queries:



[Change Theme](#)

Language

Java 8



```
4  public class Solution {  
6  public static void main(String[] args) {  
  
31      // Number of queries  
32      int q = sc.nextInt();  
33  
34      for (int i = 0; i < q; i++) {  
35  
36          int x = sc.nextInt();
```



32°C
Partly sunny



11:22 AM
8/8/2026

In each line, output the number located in y^{th} position of x^{th} line. If there is no such position, just print "ERROR!"

Sample Input

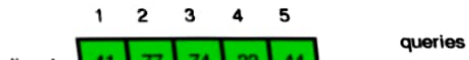
```
5
5 41 77 74 22 44
1 12
4 37 34 36 52
0
3 20 22 33
5
1 3
3 4
3 1
4 3
5 5
```

Sample Output

```
74
52
37
ERROR!
ERROR!
```

Explanation

The diagram below explains the queries:



[Download sample test cases](#)

[Suggest Edits](#)



Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✓ Sample Test case 0

Input (stdin)

```
1 12
2 ENTER John 3.75 50
3 ENTER Mark 3.8 24
4 ENTER Shafaet 3.7 35
5 SERVED
6 SERVED
7 ENTER Samiha 3.85 36
8 SERVED
9 ENTER Ashley 3.9 42
10 ENTER Maria 3.6 46
11 ENTER Anik 3.95 49
```

[Download](#)

Calvin has a math assignment at school where he has to evaluate a lot of expressions. Calvin decides to not to waste much of his time. There are M expressions overall. By looking at Susie's answers, Calvin has figured that the answers to all questions form a non decreasing sequence. He decides that all his answers are going to be between 1 and N (inclusive). He fills his answer sheet with a random non-decreasing sequence of length M where each element is between 1 and N .

Here is the part where the real problem starts for Calvin. He does not want to choose a large value of N , because, he will have a lot of options to choose from. Also, if he chooses a very small value of N , a lot of answers would become equal and the teacher will become suspicious.

If $x = \max_{1 \leq i \leq N} (f(i))$, $f(i)$ being the frequency or number of times i occurs in the sequence of M values he picked. Calvin wants to find out *expected value* of x . Help him solve the problem.

For example, if $M = 3$ & $N = 3$, the possible sequences are:

1 1 1 ($x = 3$)

1 1 2 ($x = 2$)

1 1 3 ($x = 2$)

1 2 2 ($x = 2$)

1 2 3 ($x = 3$)

Author

utkarsh1

Difficulty

Expert

Cutoff Score

50.00

Max Score

50

Submitted By

533

NEED HELP?

 [View discussions](#)
 [View editorial](#)
 [View](#)

RATE TH


 **360 Total Security**

Free antivirus to protect your security and privacy

2418. Sort the People

Easy Topics Companies Hint

You are given an array of strings `names`, and an array `heights` that consists of **distinct** positive integers. Both arrays are of length `n`.

For each index `i`, `names[i]` and `heights[i]` denote the name and height of the i^{th} person.

Return `names` sorted in **descending** order by the people's heights.

Example 1:

Input: `names = ["Mary","John","Emma"], heights = [180,165,170]`

Output: `["Mary","Emma","John"]`

Explanation: Mary is the tallest, followed by Emma and John.

Example 2:

Input: `names = ["Alice","Bob","Bob"], heights = [155,185,150]`

Output: `["Bob","Alice","Bob"]`

Explanation: The first Bob is the tallest, followed by Alice and the second Bob.

Constraints:

- `n == names.length == heights.length`

Java Auto

```
1 class Solution {
2     public String[] sortPeople(String[] names, int[] heights) {
3         int nameSize = names.length;
4         int heightsSize = heights.length;
5         String newArray[] = new String[nameSize];
6         HashMap<Integer, String> map = new HashMap<>();
7         for(int i = 0; i < nameSize; i++){
8             map.put(heights[i], names[i]);
9         }
10        List<Integer> ans = new ArrayList<>(map.keySet());
11        ans.sort((height1, height2) -> height2 - height1);
```

Saved

Ln 18, Col 2

Testcase Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

names =
["Mary", "John", "Emma"]

heights =
[180, 165, 170]

Output

2 2 3 ($x = 2$)

2 3 3 ($x = 2$)

3 3 3 ($x = 3$)

expected value of $x = 2.2$

Input Format

The first line contains an integer T which refers to the number of test cases. T lines follow, each containing 2 numbers, M and N for the corresponding test cases.

Constraints

$$1 \leq T \leq 15$$

$$1 \leq M \leq 250$$

$$1 \leq N \leq 10^9$$

Output Format

Output T lines, each containing answer to the corresponding test case. Error of upto 10^{-3} is allowed.

Sample Input

```
4
1 5
3 3
2 9
9 6
```

Sample Output

```
1.0000000000
2.2000000000
1.2000000000
4.3146853147
```

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✔ Sample Test case 0

Input (stdin)

```
1 5
2 amy 100
3 david 100
4 heraldo 50
5 aakansha 75
6 aleksa 150
```

[Download](#)

Your Output (stdout)

```
1 aleksa 150
2 amy 100
3 david 100
```



Output Format

You are not responsible for printing any output to stdout. The locked stub code in Solution will create a Checker object, use it to sort the Player array, and print each sorted element.

Sample Input

```
5
amy 100
david 100
heraldo 50
aakansha 75
aleksa 150
```

Sample Output

```
aleksa 150
amy 100
david 100
aakansha 75
heraldo 50
```

 [Download sample test cases](#)

 [Suggest Edits](#)



[Change Theme](#)

Language

Java 8



18  `class Player{`

```
1 import java.util.*;
2
3 class Checker implements Comparator<Player> {
4
5     @Override
6     public int compare(Player a, Player b) {
7
8         // Higher score first
9         if (a.score != b.score) {
10             return b.score - a.score;
11         }
12
13         // Same score -> alphabetical order
14         return a.name.compareTo(b.name);
15     }
16 }
```

```
17
18 class Player{
19     String name;
20     int score;
21
22     Player(String name, int score){
23         this.name = name;
24         this.score = score;
25     }
26 }
27
```

[Problem](#)[Submissions](#)[Leaderboard](#)[Discussions](#)[Editorial](#)

Sometimes it's better to use dynamic size arrays. Java's [Arraylist](#) can provide you this feature. Try to solve this problem using Arraylist.

You are given n lines. In each line there are zero or more integers. You need to answer a few queries where you need to tell the number located in y^{th} position of x^{th} line.

Take your input from System.in.

Input Format

The first line has an integer n . In each of the next n lines there will be an integer d denoting number of integers on that line and then there will be d space-separated integers. In the next line there will be an integer q denoting number of queries. Each query will consist of two integers x and y .

Constraints

- $1 \leq n \leq 20000$
- $0 \leq d \leq 50000$
- $1 \leq q \leq 1000$
- $1 \leq x \leq n$

Each number will fit in signed integer.

Total number of integers in n lines will not cross 10^5 .

Output Format

In each line, output the number located in y^{th} position of x^{th} line. If there is no such position, just print "ERROR!"

Sample Input

Author

[Shafaet](#)

Difficulty

Easy

Max Score

10

Submitted By

[171026](#)

NEED HELP?

[View discussions](#)[View editorial](#)[View top submissions](#)

RATE THIS CHALLENGE



MORE DETAILS

[Download problem statement](#)[Download sample test cases](#)[Suggest Edits](#)

The locked stub code prints the names of the students yet to be served in the priority order. If there are no such student, then the code prints `EMPTY`.

Sample Input 0

```
12
ENTER John 3.75 50
ENTER Mark 3.8 24
ENTER Shafaet 3.7 35
SERVED
SERVED
ENTER Samiha 3.85 36
SERVED
ENTER Ashley 3.9 42
ENTER Maria 3.6 46
ENTER Anik 3.95 49
ENTER Dan 3.95 50
SERVED
```

Sample Output 0

```
Dan
Ashley
Shafaet
Maria
```

Explanation 0

In this case, the number of events is 12. Let the name of the queue be `Q`.

- John is added to `Q`. So, it contains (John, 3.75, 50).
- Mark is added to `Q`. So, it contains (John, 3.75, 50) and (Mark, 3.8, 24).



```
1 > import java.util.ArrayList;...
4 class Student {
5     private int id;
6     private String name;
7     private double cgpa;
8
9     public Student(int id, String name, double cgpa) {
10         this.id = id;
11         this.name = name;
12         this.cgpa = cgpa;
13     }
14
15     public int getID() {
16         return id;
17     }
18
19     public String getName() {
20         return name;
21     }
22
23     public double getCGPA() {
24         return cgpa;
25     }
26 }
27
28 class Priorities {
29
```

Line: 93 Col: 2



Upload Code as File

☐ Test against custom input

Run Code

Submit Code

In computer science, a priority queue is an abstract data type which is like a regular queue, but where additionally each element has a "priority" associated with it. In a priority queue, an element with high priority is served before an element with low priority. - [Wikipedia](#)

In this problem we will test your knowledge on [Java Priority Queue](#).

There are a number of students in a school who wait to be served. Two types of events, ENTER and SERVED, can take place which are described below.

- ENTER: A student with some priority enters the queue to be served.
- SERVED: The student with the highest priority is served (removed) from the queue.

A unique id is assigned to each student entering the queue. The queue serves the students based on the following criteria (priority criteria):

1. The student having the highest Cumulative Grade Point Average (CGPA) is served first.
2. Any students having the same CGPA will be served by name in ascending case-sensitive alphabetical order.
3. Any students having the same CGPA and name will be served in ascending order of the id.

Create the following two classes:

- The Student class should implement:
 - The constructor `Student(int id, String name, double cgpa)`.
 - The method `int getID()` to return the id of the student.
 - The method `String getName()` to return the name of the student.
 - The method `double getCGPA()` to return the CGPA of the student.
- The Priorities class should implement the method `List<Student> getStudents(List<String> events)` to process all the given events and return all the students yet to be served in the priority order.

Author

Medium

Difficulty

20

Max Score

70095

Submitted By

NEED HELP?

 [View discussions](#) [View editorial](#) [View top submissions](#)

RATE THIS CHALLENGE



MORE DETAILS

 [Download problem statement](#) [Download sample test cases](#) [Suggest Edits](#)

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✓ Sample Test case 0

Compiler Message

Success

Input (stdin)

[Download](#)

```
1 4
2 1 5
3 3 3
4 2 9
5 9 6
```

Your Output (stdout)

```
1 1.0000000000
```

Explanation

For second testcase we have

Seq	Freq
111	3
112	2
113	2
122	2
123	1
133	2
222	3
223	2
233	2
333	3

$$\begin{aligned} E(x) &= \sum P_x \times x \\ &= P_1 + 2P_2 + 3P_3 \\ &= \frac{1}{10} + \frac{2 \times 6}{10} + \frac{3 \times 3}{10} \\ &= \frac{22}{10} \end{aligned}$$

[Change Theme](#)

Language Python 3



32°C
Partly sunny



11:06 AM
8/8/2026

```
1 # Enter your code here. Read input from STDIN. Print output to STDOUT
2 import sys
3 import math
4
5
6 def solve(m, n):
7     # Total number of non-decreasing sequences:
8     #  $C(n + m - 1, m)$ 
9     total = math.comb(n + m - 1, m)
10
11     # Precompute  $C(n, j)$ 
12     comb_n = [math.comb(n, j) for j in range(min(m, n) + 1)]
13
14     # Precompute:
15     #  $C(n + r - 1, r)$ , for  $r = 0..m$ 
16     comb_remaining = [0] * (m + 1)
17     comb_remaining[0] = 1
18
19     for r in range(1, m + 1):
20         comb_remaining[r] = (
21             comb_remaining[r - 1] * (n + r - 1) // r
22         )
23
24     sum_a = 0
25
26     #  $A(k)$  = number of sequences where
27     # every frequency is  $\leq k$ 
```

Line: 67 Col: 11



Upload Code as File



Test against custom input

Run Code

Submit Code

1360. Number of Days Between Two Dates

Easy Topics Companies Hint

Write a program to count the number of days between two dates.

The two dates are given as strings, their format is YYYY-MM-DD as shown in the examples.

Example 1:

Input: date1 = "2019-06-29", date2 = "2019-06-30"

Output: 1

Example 2:

Input: date1 = "2020-01-15", date2 = "2019-12-31"

Output: 15

Constraints:

- The given dates are valid dates between the years 1971 and 2100.

Seen this question in a real interview before? 1/6

Yes No

Accepted 77,082/145.2K Acceptance Rate 53.1%

Topics

Companies

Hint 1

443 42 ☆ ✎ ⓘ

16 Online

</> Code

Java Auto

```
1 import java.time.LocalDate;
2 import java.time.temporal.ChronoUnit;
3
4 class Solution {
5     public int daysBetweenDates(String date1, String date2) {
6         // parse the input strings into LocalDate objects
7         LocalDate d1 = LocalDate.parse(date1);
8         LocalDate d2 = LocalDate.parse(date2);
9
10        // calculate the absolute difference in days
11        return (int) Math.abs(ChronoUnit.DAYS.between(d1, d2));
12    }
13 }
```

Saved

Testcase Test Result

Accepted Runtime: 4 ms

Case 1 Case 2

Input

date1 =

"2019-06-29"

date2 =

"2019-06-30"

Output

2 2 3 ($x = 2$)

2 3 3 ($x = 2$)

3 3 3 ($x = 3$)

expected value of $x = 2.2$

Input Format

The first line contains an integer T which refers to the number of test cases. T lines follow, each containing 2 numbers, M and N for the corresponding test cases.

Constraints

$$1 \leq T \leq 15$$

$$1 \leq M \leq 250$$

$$1 \leq N \leq 10^9$$

Output Format

Output T lines, each containing answer to the corresponding test case. Error of upto 10^{-3} is allowed.

Sample Input

```
4
1 5
3 3
2 9
9 6
```

Sample Output

```
1.0000000000
2.2000000000
1.2000000000
4.3146853147
```